

Aula 1

Self-Attention: a mecânica central

Da motivação em processamento de texto até a forma matricial do self-attention — construindo, formula por formula, as equações 12.1 a 12.8.



O que vamos construir hoje

Cada fórmula é uma peça — vamos parar em cada uma antes de seguir para a próxima.

12.1	Motivação: por que texto quebra o que já sabemos	3 observações	12.2.3	Resumo do self-attention	propriedades
12.2.1	Values: computando e ponderando	eq. 12.1 – 12.3	12.2.4	Forma matricial	eq. 12.6 – 12.8
12.2.2	Attention weights: queries, keys, softmax	eq. 12.4 – 12.5	→	Fechamento e próximos passos	o que fica para depois

Por que texto quebra o que já sabemos

O capítulo abre com uma passagem de exemplo — vale ler com atenção antes das três observações que ela motiva.

*"The restaurant refused to serve me a ham sandwich because **it** only cooks vegetarian food. In the end, they just gave me two slices of bread. **Their** ambiance was just as good as the food and service."*

UDL, SEÇÃO 12.1

- 01 Dimensionalidade explode. 37 palavras × embedding de 1024 = 37.888 valores — inviável para uma rede densa relacionar tudo com tudo.
- 02 Comprimento variável. Cada sequência tem tamanho diferente; uma rede densa tem número fixo de entradas.
- 03 Ambiguidade referencial. "it" refere-se a "restaurant", não a "sandwich" — isso não está na sintaxe, está no significado, e depende do *conteúdo* das palavras, não da posição.

Dois requisitos de design

As três observações anteriores levam a duas exigências que qualquer arquitetura para texto precisa satisfazer.

REQUISITO (I)

Compartilhamento de parâmetros ao longo da sequência — como convolução faz espacialmente em imagens. A rede não deve "reaprender" o que significa *dog* a cada posição.

REQUISITO (II)

Conexões entre palavras cuja força depende do **conteúdo** das palavras — algo que convolução, com kernel fixo, não faz.

O transformer resolve os dois simultaneamente via **dot-product self-attention**. É isso que vamos construir, peça por peça, a partir de agora.

O primeiro passo: computar os values

Antes de tudo, um lembrete de como é uma camada densa comum — depois, a primeira equação do self-attention.

EQ. 12.1 – camada densa padrão

$$f[\mathbf{x}] = \text{ReLU}[\boldsymbol{\beta} + \boldsymbol{\Omega}\mathbf{x}]$$

EQ. 12.2 – value do token m

$$\mathbf{v}_m = \boldsymbol{\beta}_v + \boldsymbol{\Omega}_v \mathbf{x}_m$$

$\boldsymbol{\Omega}_v$ e $\boldsymbol{\beta}_v$ são os mesmos para todo m — o mesmo "kernel" desliza pela sequência inteira, exatamente como convolução desliza pela imagem.

"gato dorme muito" — três tokens, D = 4

m e n são índices de **posição na sequência** (não de features). Cada token é um vetor em $\mathbb{R}^D$;

$$\mathbf{x}_1 = \begin{bmatrix} 0.2 \\ 0.9 \\ 0.1 \\ 0.4 \end{bmatrix}$$

"gato" (m=1)

$$\mathbf{x}_2 = \begin{bmatrix} 0.7 \\ 0.1 \\ 0.8 \\ 0.2 \end{bmatrix}$$

"dorme" (m=2)

$$\mathbf{x}_3 = \begin{bmatrix} 0.3 \\ 0.5 \\ 0.2 \\ 0.9 \end{bmatrix}$$

"muito" (m=3)

$$\mathbf{v}_1 = \beta_v + \Omega_v \mathbf{x}_1, \quad \mathbf{v}_2 = \beta_v + \Omega_v \mathbf{x}_2, \quad \mathbf{v}_3 = \beta_v + \Omega_v \mathbf{x}_3$$

A mesma função $f(\mathbf{x}) = \beta_v + \Omega_v \mathbf{x}$ aplicada três vezes — pesos idênticos, resultados diferentes porque as entradas diferem.

Roteamento: a soma ponderada

Cada saída é uma combinação diferente dos mesmos values — pense nisso como "roteamento" (Fig. 12.1 do livro).

EQ. 12.3

$$\text{sa}_n[\mathbf{x}_1, \dots, \mathbf{x}_N] = \sum_{m=1}^N a[\mathbf{x}_m, \mathbf{x}_n] \mathbf{v}_m$$

Os pesos $a[\mathbf{x}_m, \mathbf{x}_n]$ são não-negativos e somam 1 (para cada n fixo).

$$0.1 \cdot v_1 \text{ (gato)} + 0.7 \cdot v_2 \text{ (dorme)} + 0.2 \cdot v_3 \text{ (muito)} = \text{sa}_2 \text{ ("dorme")}$$

Duas escalas de crescimento distintas

Um detalhe que volta na seção 12.9 (sequências longas)

Ω_v – transformação dos values

$$\Omega_v \in \mathbb{R}^{D \times D}$$

Escala **linearmente** com N — o mesmo Ω_v de tamanho fixo é reaplicado a cada token.

$a[x_m, x_n]$ – pesos de atenção

$$N \times N \text{ pesos}$$

Cresce **quadraticamente** em N — um peso por par ordenado de tokens, independente de D.

De onde vêm os pesos: queries e keys

Duas transformações lineares novas — mais um hypernetwork: um ramo da rede computa os pesos que o outro ramo usa.

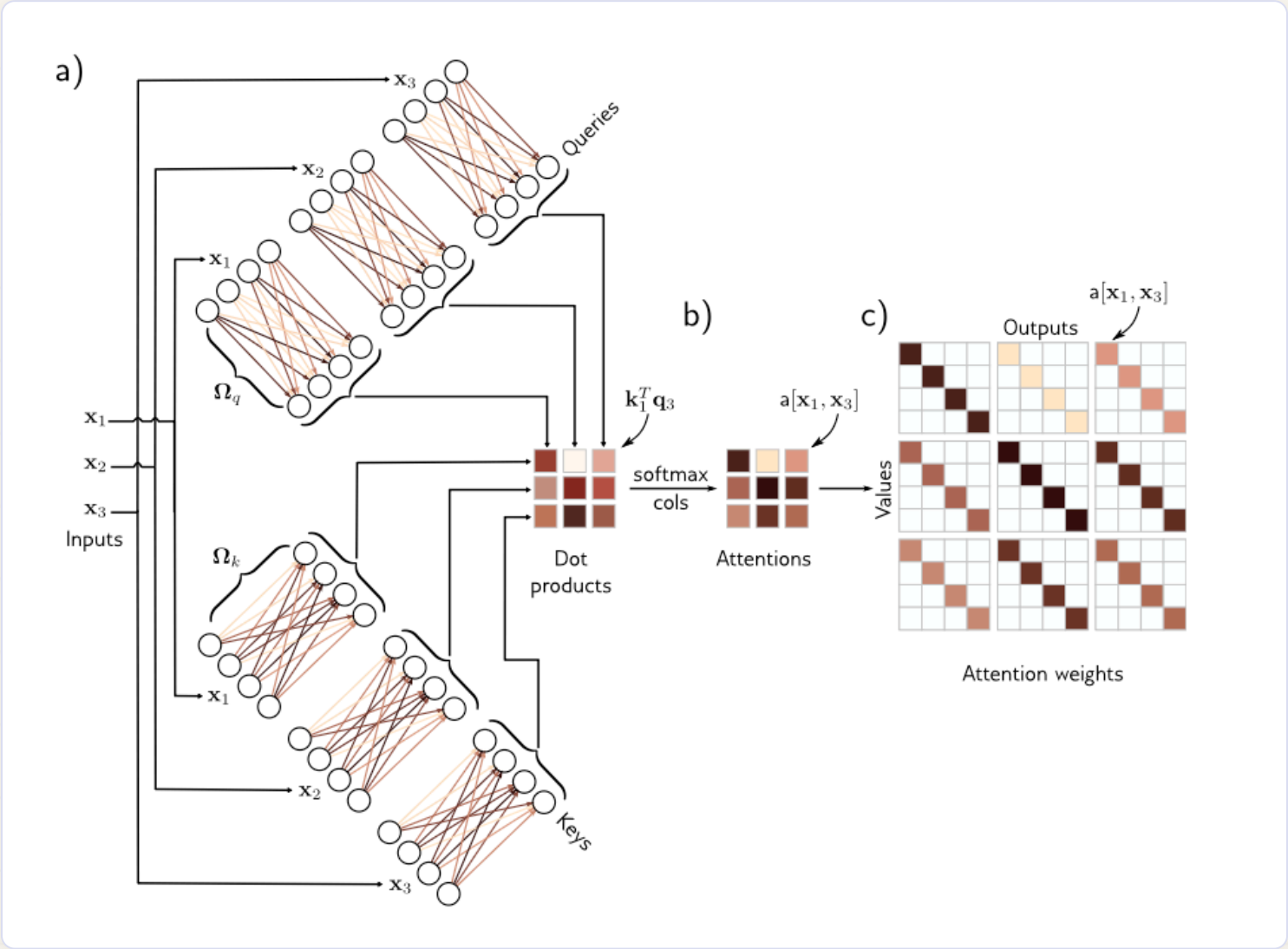
EQ. 12.4

$$\mathbf{q}_n = \beta_q + \Omega_q \mathbf{x}_n \qquad \mathbf{k}_m = \beta_k + \Omega_k \mathbf{x}_m$$

Query "pergunta" o que este token procura; key "anuncia" o que cada token oferece — nomes herdados de information retrieval.

Fig. 12.3 — Computing attention weights

Queries e keys calculadas em paralelo → produtos escalares → softmax por coluna → pesos de atenção roteando os values.



PRINCE, UNDERSTANDING DEEP LEARNING – CAP. 12, FIG. 12.3

Softmax: da similaridade ao peso

O produto escalar mede similaridade; o softmax faz as keys competirem entre si por atenção.

EQ. 12.5

$$a[\mathbf{x}_m, \mathbf{x}_n] = \text{softmax}_m[\mathbf{k}_{\bullet}^T \mathbf{q}_n] = \frac{\exp[\mathbf{k}_m^T \mathbf{q}_n]}{\sum_{m'=1}^N \exp[\mathbf{k}_{m'}^T \mathbf{q}_n]}$$

Query e key precisam ter a **mesma dimensão** — mas essa dimensão pode diferir da dimensão dos values.

A query de "dorme" contra todas as keys

Continuando nosso exemplo: a query do token central compara-se com a key de cada token da sequência (incluindo ele mesmo).



Por que este mecanismo funciona

Juntando os pedaços: linear duas vezes (values e sua combinação), mas não-linear no conjunto — por causa do produto escalar e do softmax.

parâmetros compartilhados

$$\phi = \{\beta_v, \Omega_v, \beta_q, \Omega_q, \beta_k, \Omega_k\}$$

Independente de N — a mesma rede lida com sequências de qualquer tamanho.

SATISFAZ REQUISITO (I)

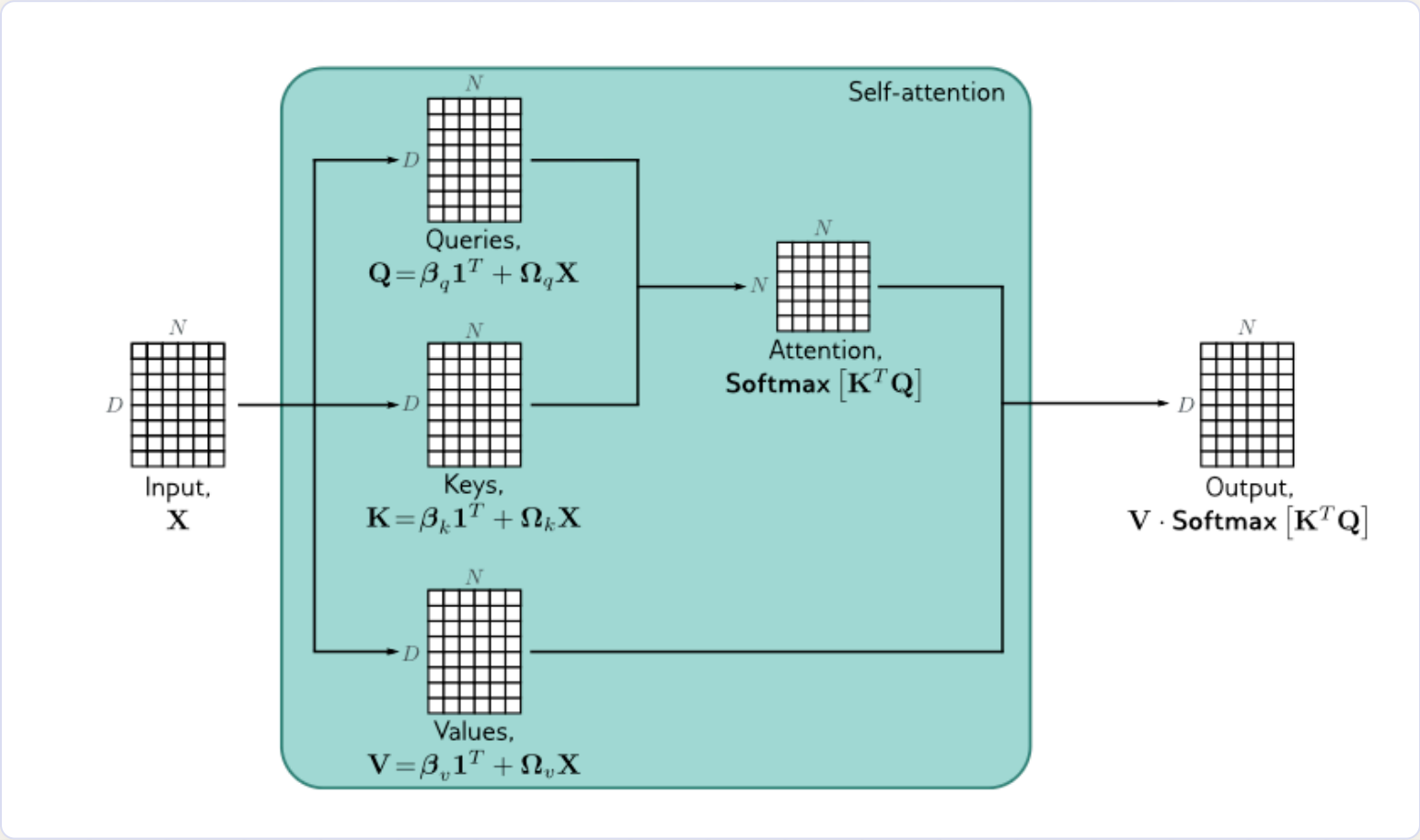
ϕ é fixo, independente de N — parâmetros compartilhados ao longo da sequência.

SATISFAZ REQUISITO (II)

A força de cada conexão (os pesos a) depende do conteúdo dos próprios tokens.

Fig. 12.4 — Self-attention in matrix form

O mecanismo inteiro, com os N tokens empilhados como colunas de X — a forma que roda de fato em GPU.



PRINCE, UNDERSTANDING DEEP LEARNING – CAP. 12, FIG. 12.4

Empilhando tudo em matrizes

Os N inputs viram colunas de uma matriz $\mathbf{X} \in \mathbb{R}^{(D \times N)}$ — é essa forma que roda de fato em GPU.

EQ. 12.6

$$\begin{aligned}\mathbf{V}[\mathbf{X}] &= \boldsymbol{\beta}_v \mathbf{1}^T + \boldsymbol{\Omega}_v \mathbf{X} \\ \mathbf{Q}[\mathbf{X}] &= \boldsymbol{\beta}_q \mathbf{1}^T + \boldsymbol{\Omega}_q \mathbf{X} \\ \mathbf{K}[\mathbf{X}] &= \boldsymbol{\beta}_k \mathbf{1}^T + \boldsymbol{\Omega}_k \mathbf{X}\end{aligned}$$

$\mathbf{1}$ é um vetor $N \times 1$ de uns — só um truque para "espalhar" o bias em todas as N colunas.

A cadeia completa, eq. 12.1 $\rightarrow$ 12.8

De uma camada densa comum até a forma matricial do self-attention — em oito passos.

12.1	$f[\mathbf{x}] = \text{ReLU}[\boldsymbol{\beta} + \boldsymbol{\Omega}\mathbf{x}]$	camada densa (lembrete)
12.2	$\mathbf{v}_m = \boldsymbol{\beta}_v + \boldsymbol{\Omega}_v\mathbf{x}_m$	values, pesos compartilhados
12.3	$\text{sa}_n = \sum_m a[\mathbf{x}_m, \mathbf{x}_n] \mathbf{v}_m$	roteamento / soma ponderada
12.4	$\mathbf{q}_n = \boldsymbol{\beta}_q + \boldsymbol{\Omega}_q\mathbf{x}_n, \mathbf{k}_m = \boldsymbol{\beta}_k + \boldsymbol{\Omega}_k\mathbf{x}_m$	queries e keys
12.5	$a = \text{softmax}_m[\mathbf{k}_\bullet^T \mathbf{q}_n]$	pesos de atenção
12.6	$\mathbf{V}, \mathbf{Q}, \mathbf{K}$ em forma matricial	empilhando em X
12.8	$\text{Sa}[\mathbf{X}] = \mathbf{V} \cdot \text{Softmax}[\mathbf{K}^T \mathbf{Q}]$	self-attention completo

O que fica de pé até aqui

Self-attention "puro": um mecanismo não-linear, com parâmetros compartilhados, que roteia values com pesos dependentes do conteúdo — resolvendo os dois requisitos da motivação inicial.

Ficou de fora, de propósito: **positional encoding**, **scaled dot-product** (o porquê do $\sqrt{D_q}$) e **multi-head attention** — as três extensões que tornam este mecanismo utilizável na prática.